자바 IO 아키텍처: 데이터 물류 네트워크의 이해

바이트 스트림부터 객체 직렬화까지, 자바 입출력 시스템의 시각적 가이드

Context Note: 하드디스크의 메모리 사이를 흐르는 데이터의 원뿌리 통제

NotebookLM

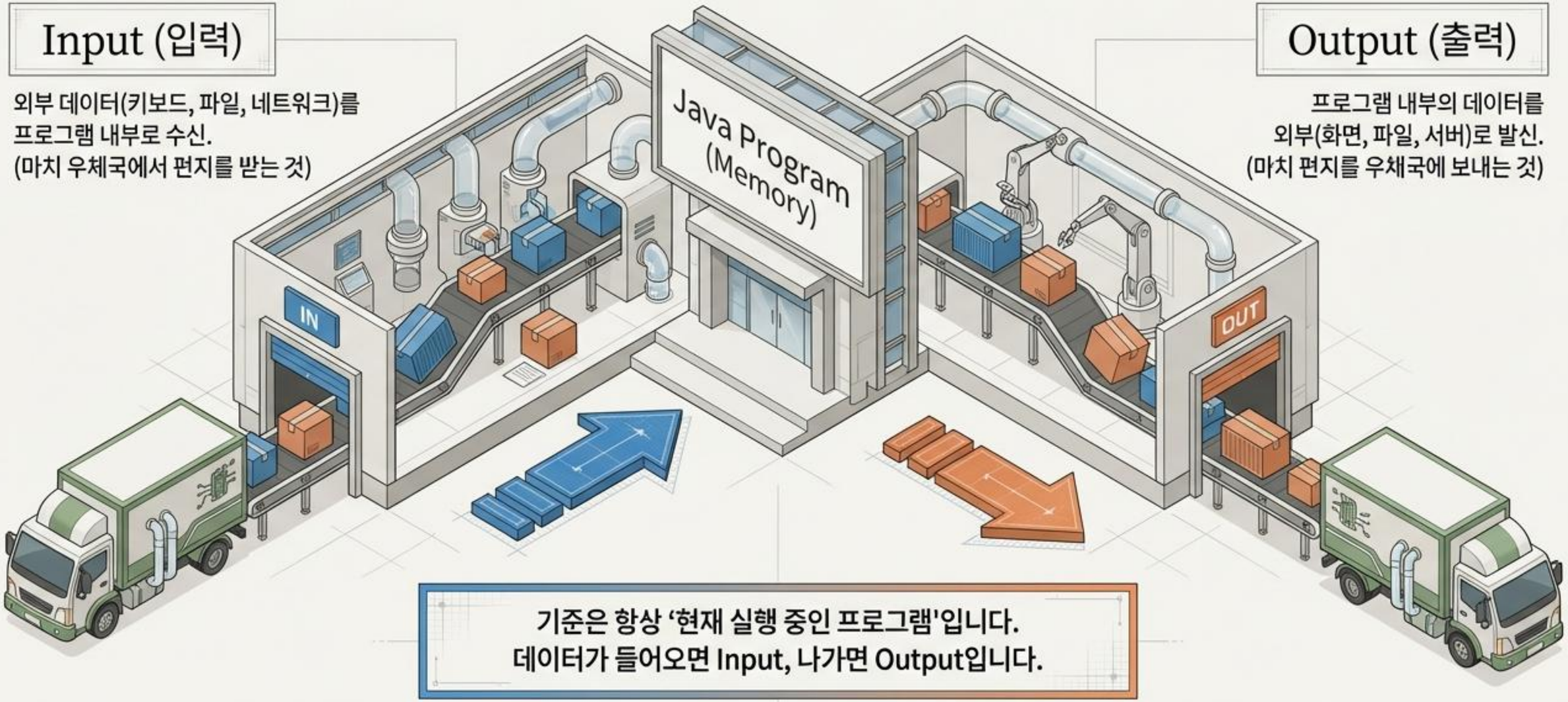
IO의 본질: 디지털 우체국

Input (입력)

외부 데이터(키보드, 파일, 네트워크)를
프로그램 내부로 수신.
(마치 우체국에서 편지를 받는 것)

Output (출력)

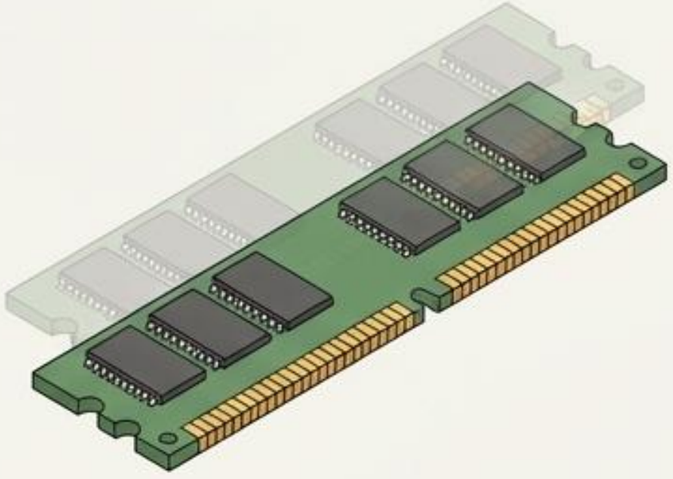
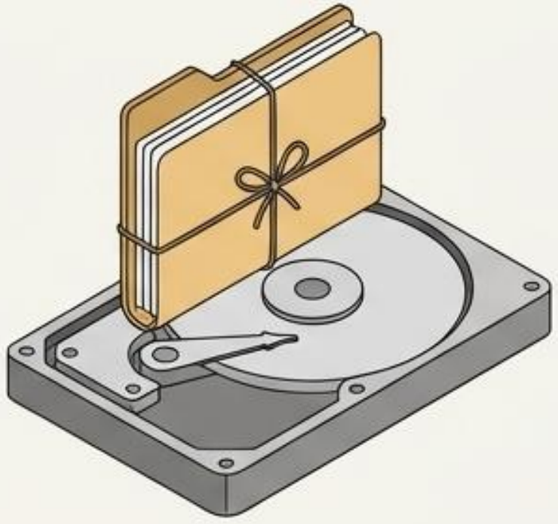
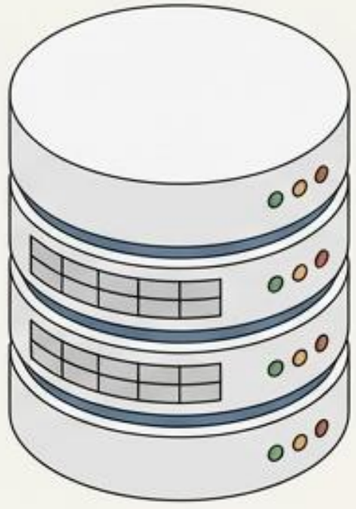
프로그램 내부의 데이터를
외부(화면, 파일, 서버)로 발신.
(마치 편지를 우체국에 보내는 것)



기준은 항상 '현재 실행 중인 프로그램'입니다.
데이터가 들어오면 Input, 나가면 Output입니다.

NotebookLM

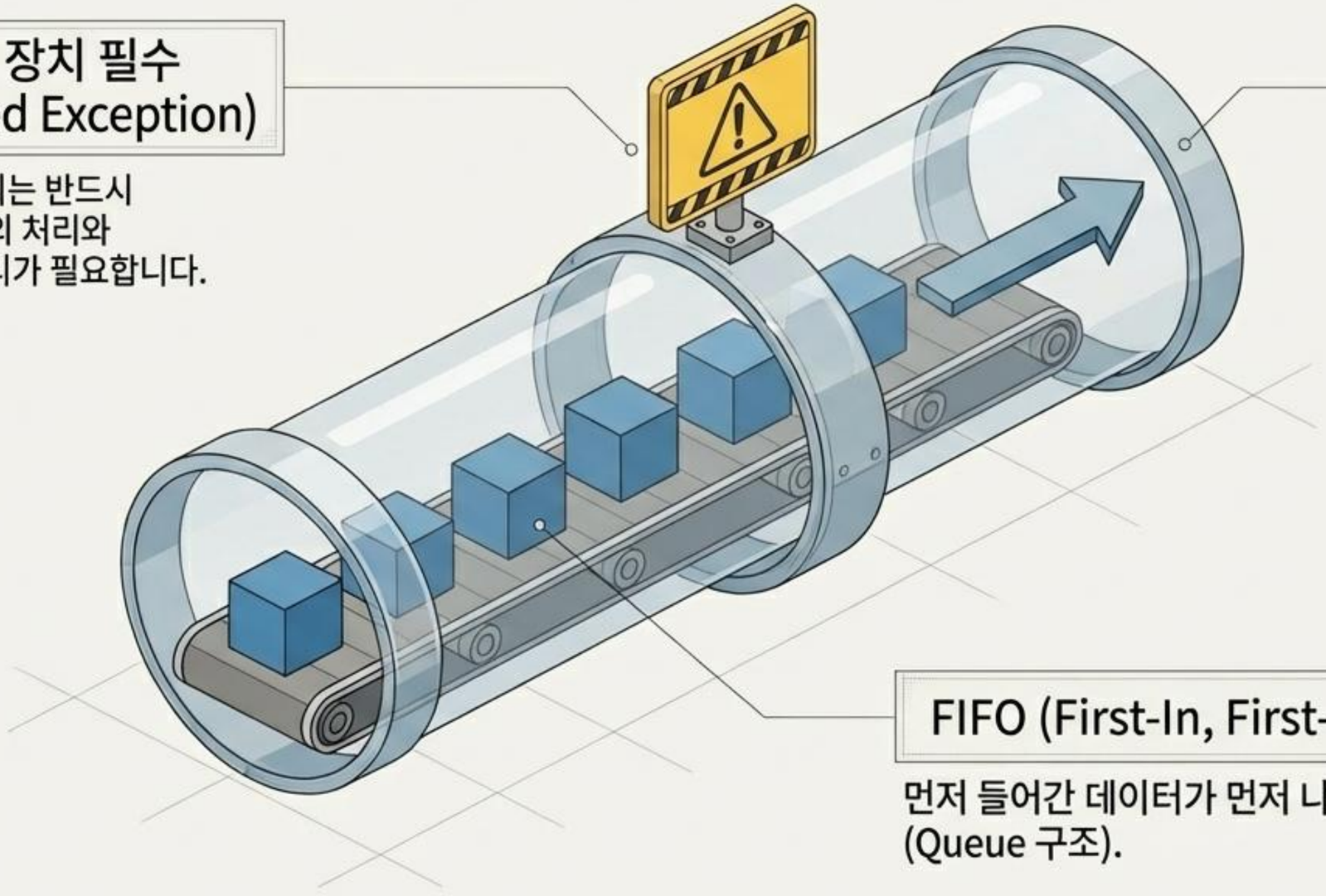
데이터의 영속성: 왜 파일 IO가 필요한가?

Memory (메모리)	File IO (파일)	RDBMS (데이터베이스)
		
특징: 휘발성 (Volatile) **속도:** 매우 빠름 **한계:** 프로그램 종료 시 모든 데이터 증발	**특징:** 영구 보존 (Permanent) **속도:** 보통 **한계:** 구조적 데이터 검색 어려움, 구분자 필요	**특징:** 영구 보존 및 구조화 **속도:** 네트워크 의존 **한계:** 별도의 DB 서버 구축 필요

자바 IO는 휘발성 메모리의 한계를 극복하고, 데이터를 물리적 하드디스크에 영구적으로 기록하는 첫 걸음입니다.

NotebookLM

스트림(Stream): 단방향 데이터 컨베이어 벨트



안전 장치 필수 (Checked Exception)

java.io 패키지는 반드시 try-catch 예외 처리와 **close()** 마무리가 필요합니다.

단방향 통신 (One-Way)

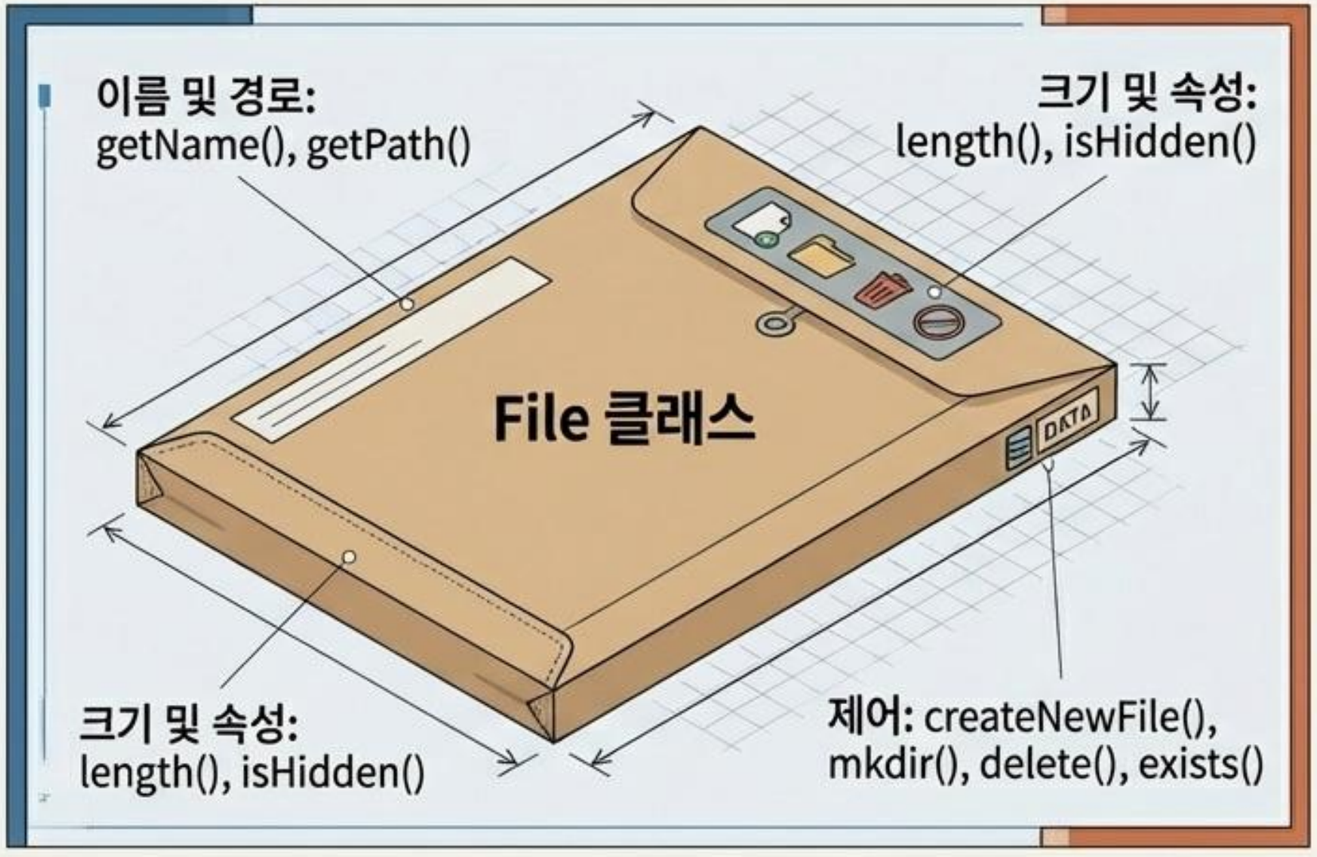
입력과 출력을 위한 빨대(통로)는 따로 존재합니다. 양방향 통신을 위해서는 두 개의 스트림이 필요합니다.

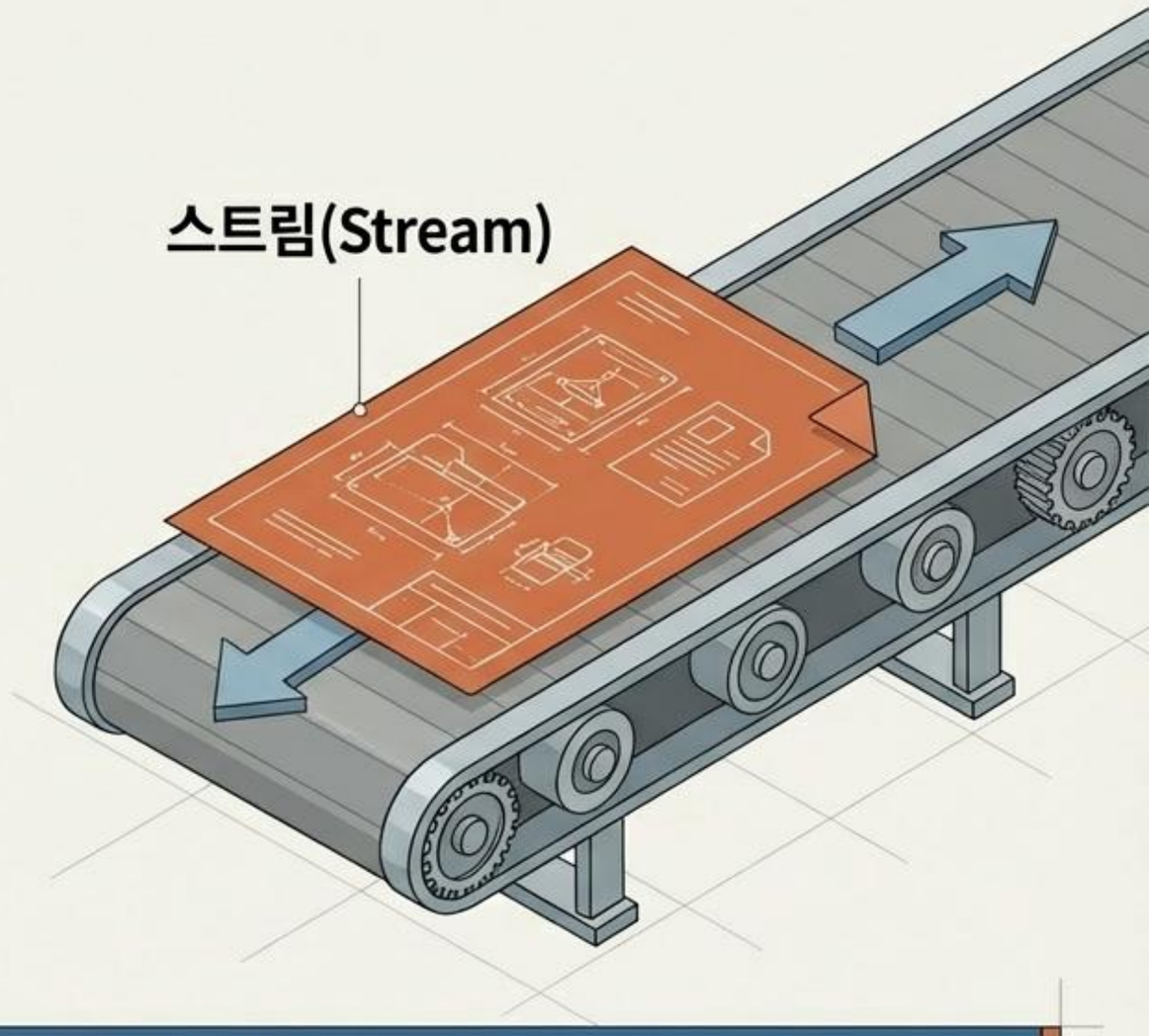
FIFO (First-In, First-Out)

먼저 들어간 데이터가 먼저 나옵니다 (Queue 구조).

NotebookLM

File 클래스: 편지가 아닌 '봉투'





File 객체는 파일이나 폴더의 **정보(메타데이터)**를 다룹니다.
파일 내부의 실제 데이터를 읽고 쓰려면 반드시 **스트림(Stream)**을 **연결**해야 합니다.

NotebookLM

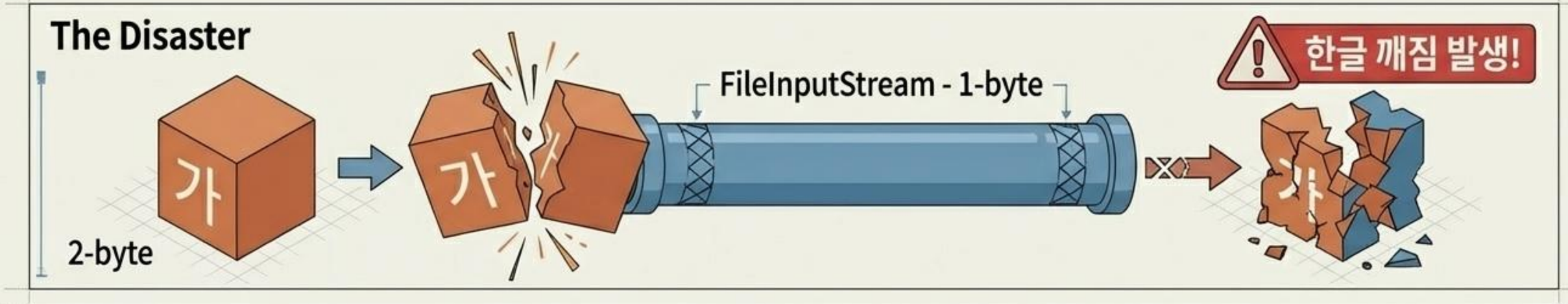
데이터의 두 갈래: 바이트 vs 문자 스트림

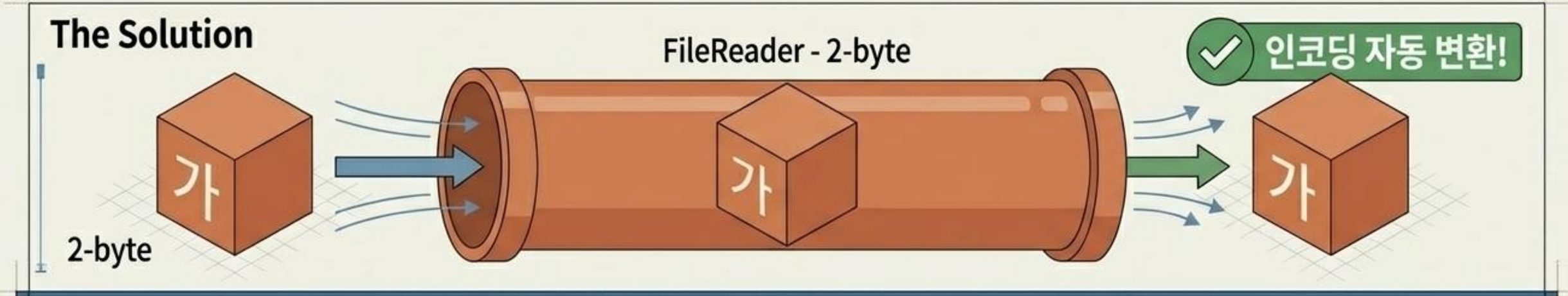
	바이트 스트림	문자 스트림	
데이터 단위	1-Byte (원시 데이터)	데이터 단위	최상위 클래
최상위 클래스	InputStream / OutputStream	2-Byte (유니코드 중심)	Reader / Writer
대표 구현체	FileInputStream / FileOutputStream		FileReader / FileWriter
주요 용도	모든 파일 (이미지, 동영상, 텍스트 포함)		오직 텍스트 파일 (.txt, .log)

바이트 스트림은 범용 화물선, 문자 스트림은 문서 전용 특급 우편입니다.

NotebookLM

왜 문자 스트림을 써야 할까? (인코딩 브레이커)

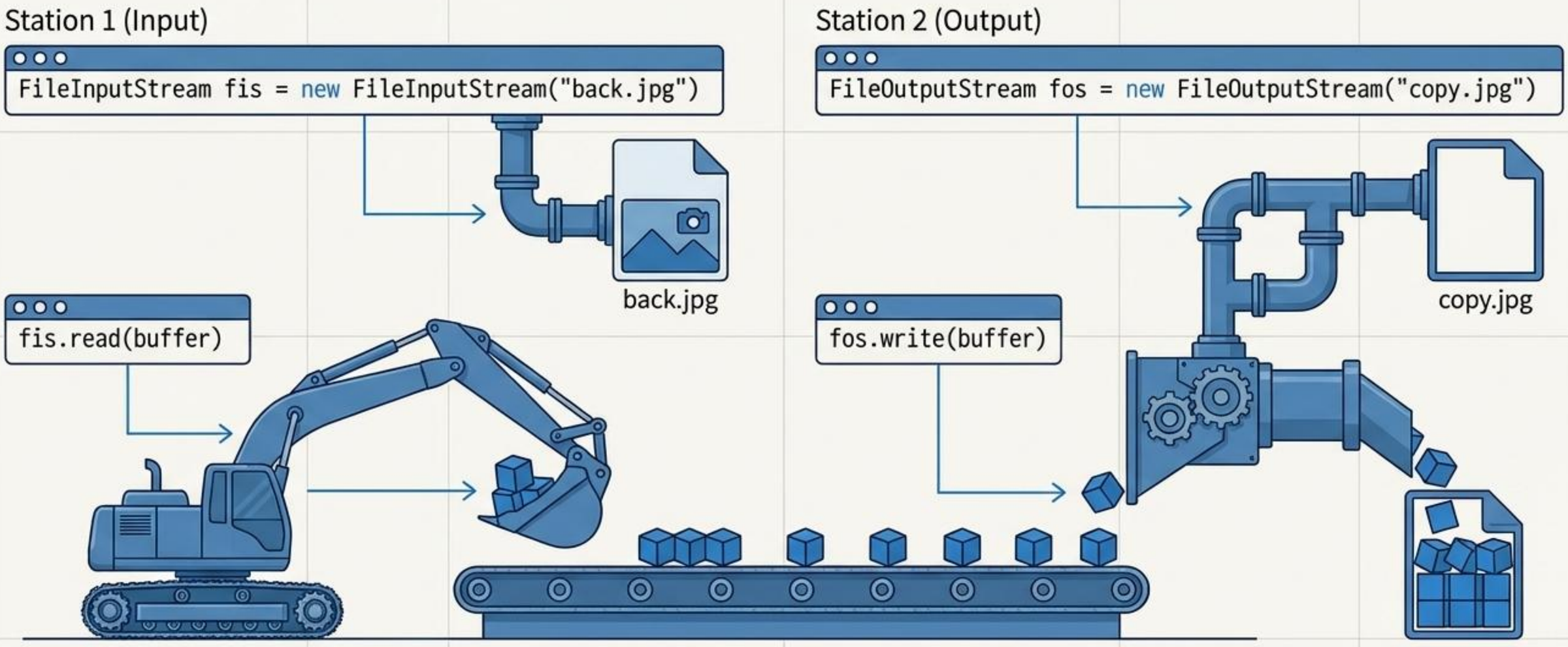




영문과 숫자는 1바이트지만, 한글은 글자당 2바이트를 차지합니다.
텍스트 데이터를 1바이트 스트림으로 읽으면 데이터가 반토막 나며 훼손됩니다.

NotebookLM

바이트 스트림 (Byte Stream) 작동 설계도



핵심 메서드: .getBytes() – 문자열을 바이트 스트림에 태우기 위해 원시 바이트 배열로 분해하는 필수 과정.

NotebookLM

문자 스트림 (Character Stream) 작동 설계도

Station 1 (Input)

```
FileReader fr = new FileReader("sample.txt")
```

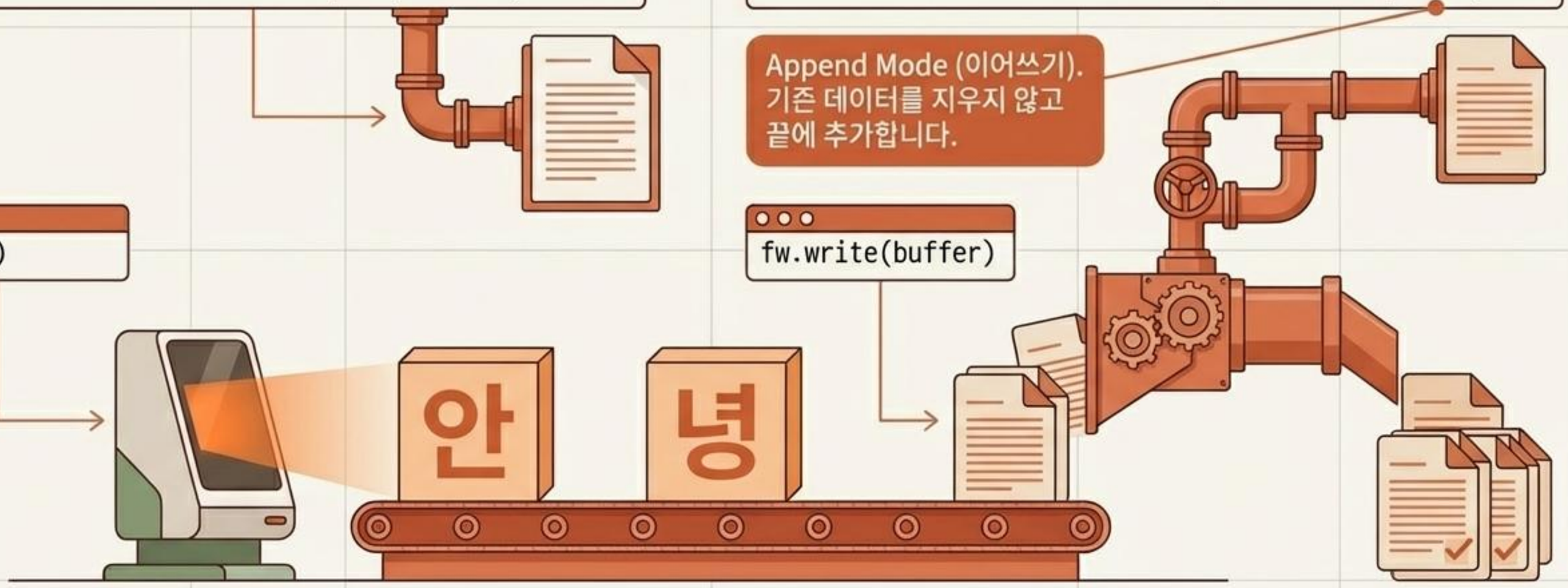
Station 2 (Output)

```
FileWriter fw = new FileWriter("board.txt", true)
```

Append Mode (이어쓰기).
기존 데이터를 지우지 않고
끝에 추가합니다.

```
fr.read()
```

```
fw.write(buffer)
```



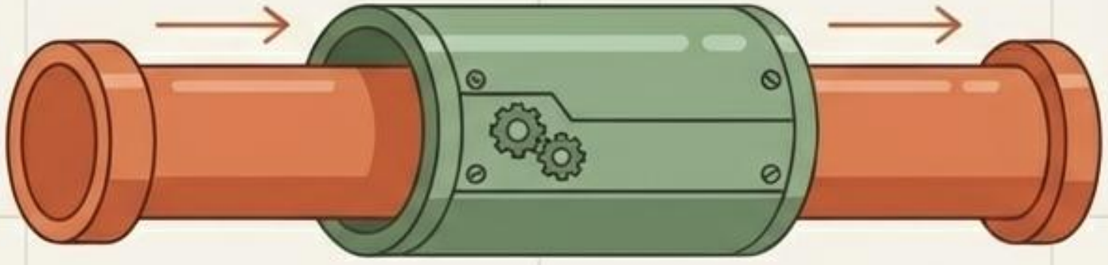
내부적으로 **인코딩과 디코딩**을 **자동으로 처리**하여 개발자가 문자열(String)을 그대로 던질 수 있게 해줍니다.

NotebookLM

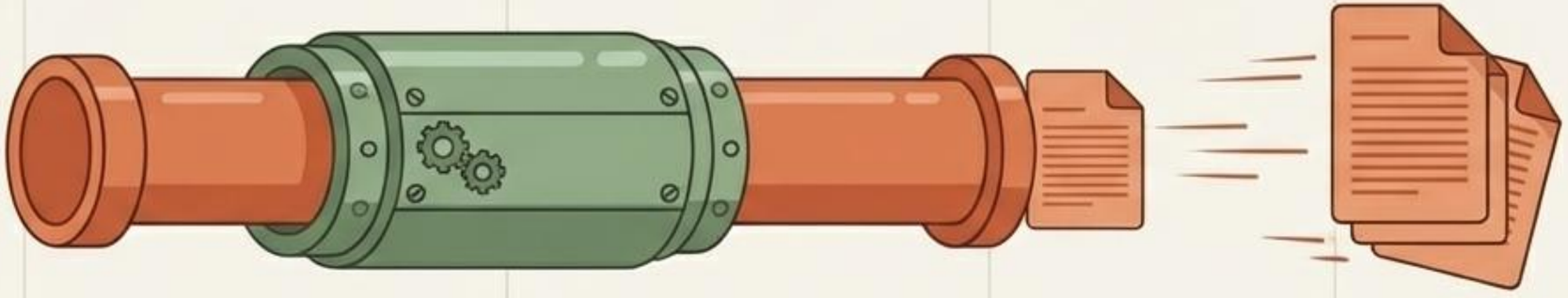
보조 스트림: 파이프의 성능을 높이는 '업그레이드 파츠'

1. 기반 스트림 (예: FileReader)
기본적인 데이터 통로 구축

2. 보조 스트림 장착
(예: BufferedReader)
스스로 입출력은 불가능, 기반 스트림을 감싸서(Wrap) 성능 강화



3. 조립 완료
속도 향상 및 새로운 편의 기능 획득

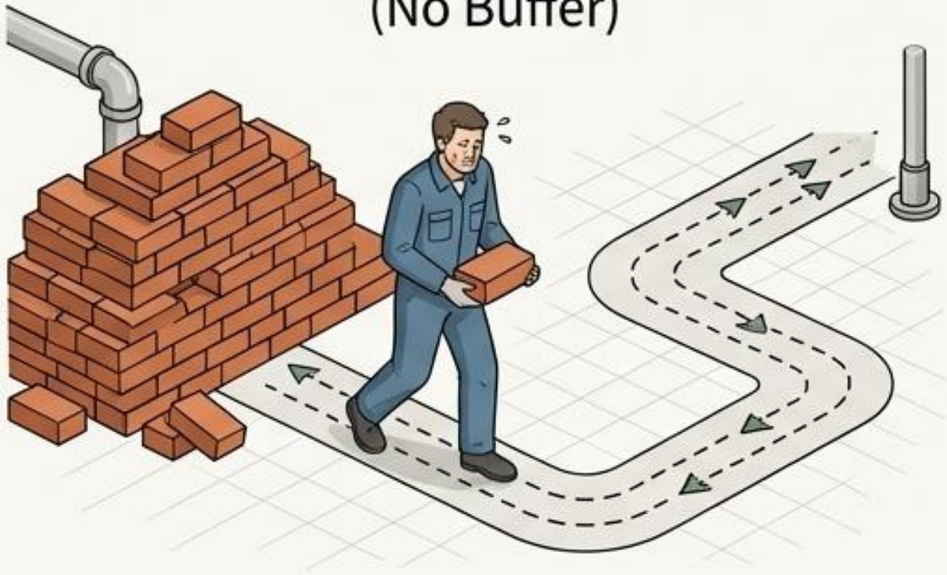


보조(필터) 스트림은 스스로 데이터를 파일에 읽고 쓸 수 없습니다.
반드시 기반 스트림을 감싸서(Wrap) 속도를 높이거나 새로운 기능을 부여하는 역할만 수행합니다.

NotebookLM

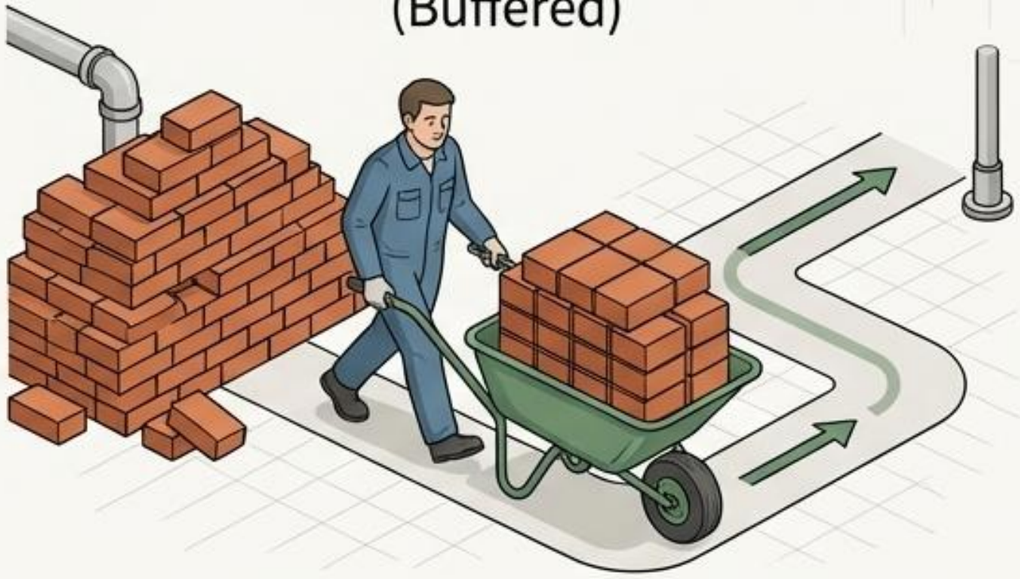
속도와 편의성의 핵심: Buffered Stream

(No Buffer)



1바이트/문자씩 이동 (FileReader).
디스크 접근 잦음 = 심각한 속도 저하.

(Buffered)



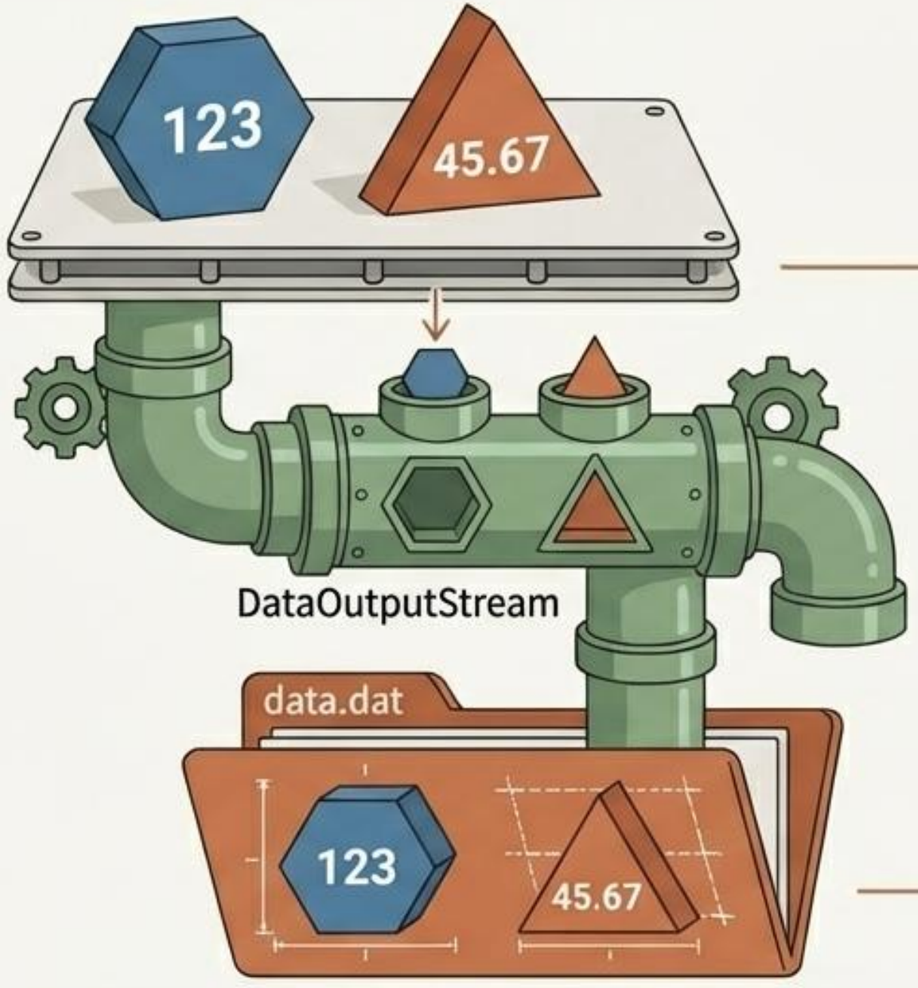
버퍼에 모아서 한 번에 이동 (BufferedReader).
메모리 활용 = 극대화된 속도 최적화.

```
BufferedReader br = new BufferedReader(new FileReader("data.txt"));
```

강력한 추가 기능: .readLine() 메서드 활성화! 문자열을 한 줄씩(Line by Line) 통째로 읽어올 수 있습니다.

NotebookLM

원시 데이터의 원형 보존: Data Stream




```
dos.writeInt(123);
dos.writeDouble(45.67);
```

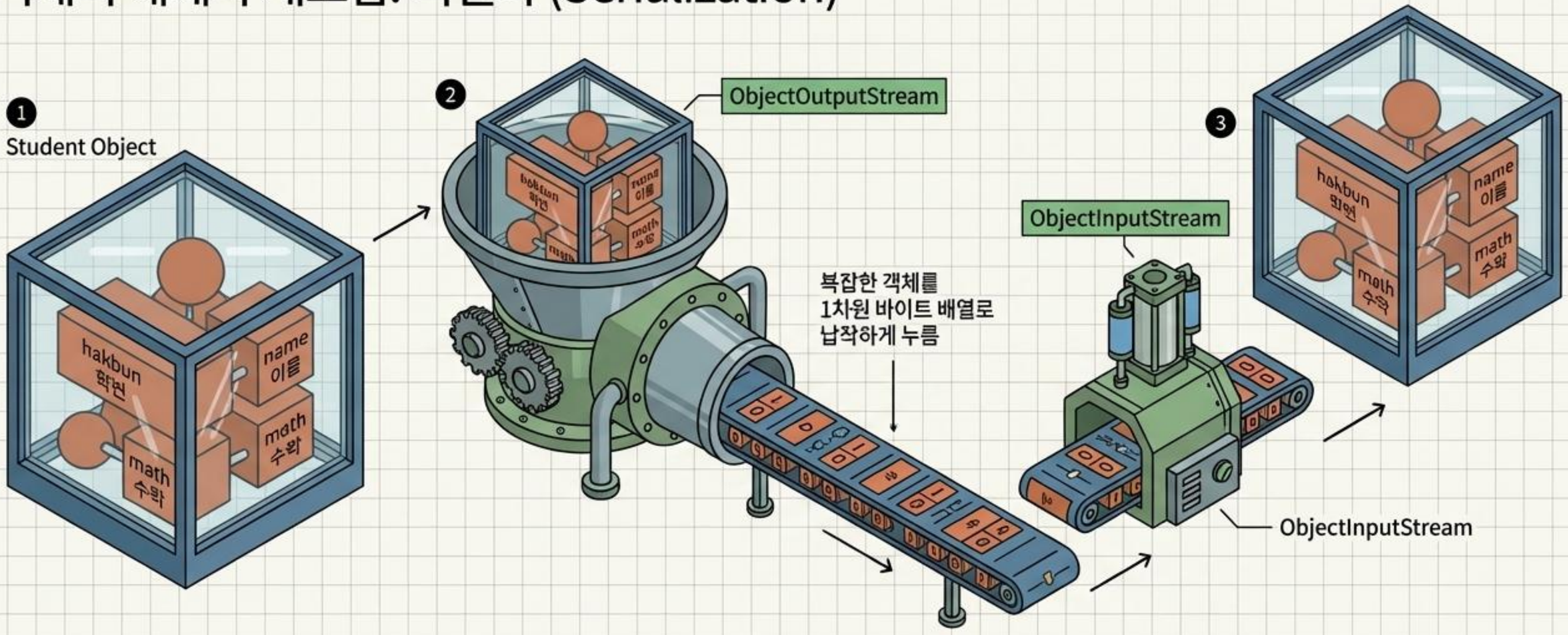
```
dis.readInt();
dis.readDouble();
```

└ 실수형 구조로 정확히 복원
(형 변환 불필요)

자바의 기본 자료형(int, double, boolean 등)을 문자열로 변환하지 않고,
메모리에 저장된 원시 데이터 형태 그대로 파일에 기록하고 읽어옵니다.

NotebookLM

객체의 해체와 재조립: 직렬화 (Serialization)

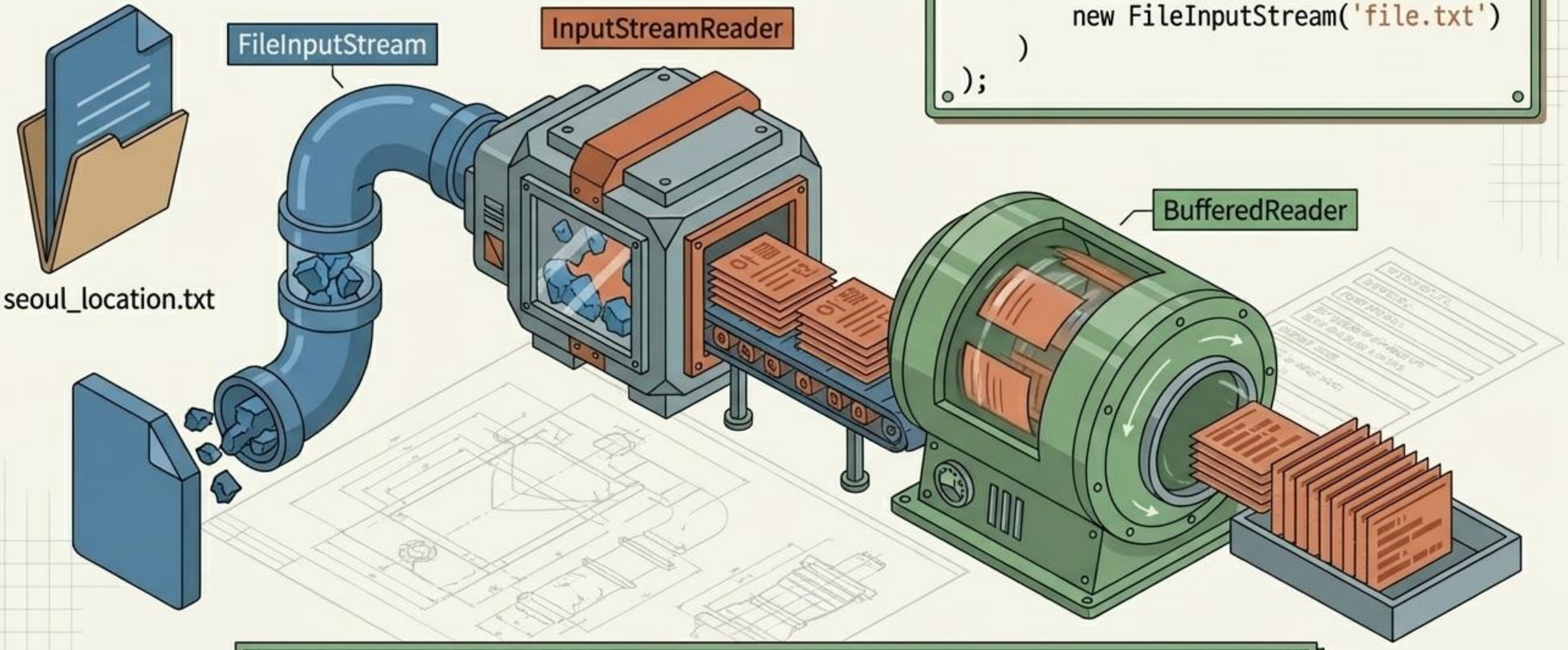


직렬화의 본질: 메모리에 떠있는 복잡한 객체(병렬 데이터)를 파일 전송을 위해 한 줄의 바이트 스트림으로 납작하게 펴는 작업.
필수 조건: 전송할 객체 클래스는 반드시 'implements Serializable' 선언이 필요합니다.

NotebookLM

설계 패턴의 완성: 왜 스트림을 겹쳐 쓸까?

```
BufferedReader br = new BufferedReader(
    new InputStreamReader(
        new FileInputStream('file.txt')
    )
);
```



파일 연결(바이트) ➡ 한글 번역(문자) ➡ 속도 향상(버퍼).
객체 지향의 유연성을 극대화한 조립식 설계(Decorator Pattern)입니다.

NotebookLM

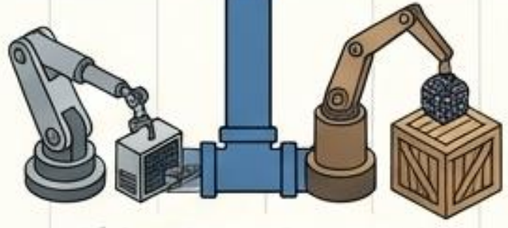
자바 IO 아키텍처: 실전 선택 가이드

Q: 다루려는 데이터가 텍스트(문자)인가요?

NO (이미지, 동영상, 압축파일, 원시 데이터)

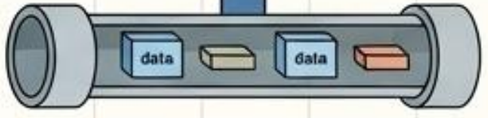
➜ FileInputStream / FileOutputStream

객체를 통째로 저장해야 하는가?



ObjectInputStream / OutputStream

원시 타입(int, double)을 다루는가?



DataInputStream / OutputStream

YES (로그, TXT, CSV 파일, 한글 텍스트)

➜ FileReader / FileWriter

빠른 처리와 한 줄 읽기(readLine)가 필요한가?

BufferedReader / BufferedWriter

스트림은 파이프, 보조 스트림은 모터입니다. 목적에 맞는 파츠를 조립하여 완벽한 데이터 물류망을 구축하세요.

NotebookLM